A RISC-V SoC with a Hardware Accelerator on the Tang Nano 9K

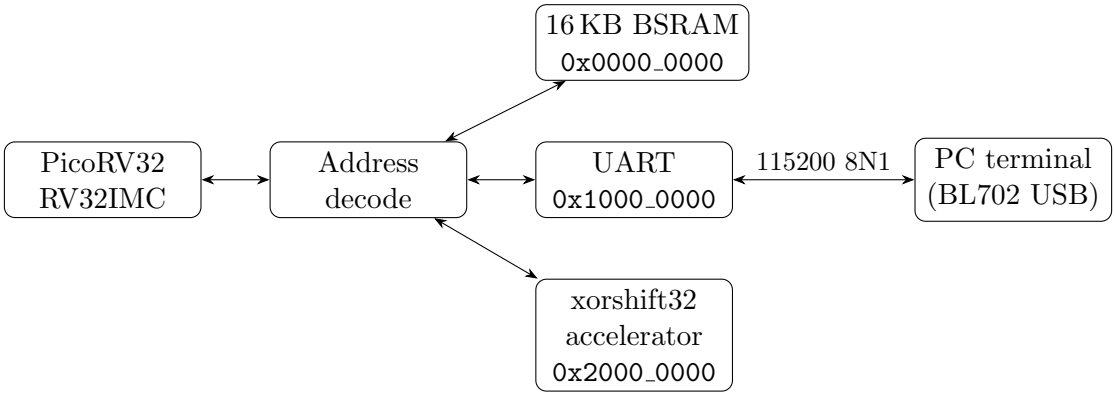
Parth Sinha

October 10, 2026

Abstract

A small system-on-chip built entirely with open-source tools on a Sipeed Tang Nano 9K (Gowin GW1NR-9). A PicoRV32 RISC-V core runs C firmware from 16 KB of block RAM, talks to a PC over a hardware UART, and offloads a pseudo-random-number generator to a memory-mapped accelerator. On the real board the accelerator ran 1000 xorshift32 steps in 1066 cycles against 50015 in software, a 46× reduction. Source: https://github.com/sinhaparth5/tangnano9k-riscv-soc.

1 Architecture



The CPU uses PicoRV32's native memory interface with one wait state on every access. The 27 MHz board oscillator is used directly, with no PLL.

1.1 Memory map

Address	Register	Access
`0x0000_0000`–`0x0000_3FFF`	RAM (code, data, stack)	R/W
`0x1000_0000`	UART data: write sends a byte, read pops the RX FIFO	R/W
`0x1000_0004`	UART status: bit 0 TX busy, bit 1 RX not empty	R
`0x2000_0000`	Accelerator `X`: PRNG state and result	R/W
`0x2000_0004`	`N`: number of steps	W
`0x2000_0008`	`CTRL`: bit 0 starts a run	W
`0x2000_000C`	`STATUS`: bit 0 busy	R

1.2 Accelerator

The accelerator advances a 32-bit xorshift state ($x \oplus= x \ll 13$; $x \oplus= x \gg 17$; $x \oplus= x \ll 5$) once per clock for N clocks. The CPU writes `X` and `N`, sets `CTRL`, polls `STATUS`, then reads `X`.

2 Toolchain

Yosys (synthesis), nextpnr-himbaechel (place and route), Apicula `gowin_pack` (bitstream), openFPGALoader (programming), Icarus Verilog (simulation) and `riscv64-unknown-elf-gcc` with `-march=rv32imc -mabi=ilp32` for firmware. One Makefile drives everything: `make PHASE=p4_accel [sim|prog|flash]`. Firmware is compiled, converted to a one-word-per-line hex file with `objcopy` and `od`, and loaded into the RAM at synthesis time with `$readmemh`.

3 Phases and verification

Phase	Design	Verified
1	LED counter	On board: six LEDs count in binary
2	UART echo	Simulation loopback test; on board: typed characters echo
3	PicoRV32, RAM, UART	Simulation runs the real firmware and checks the prompt; on board: banner and prompt appear
4	Accelerator and shell	Simulation checks `seed`, `rand`, `peek`; on board: `rand`, `seed`, `bench`, `peek`

The Phase 4 shell offers `help`, `rand [n]`, `seed <hex>`, `bench [n]`, `peek <addr>`, `poke <addr> <val>` and `cycles`. Results were checked against an independent Python implementation: five steps from seed 1 give `8ef917d1`, ten steps give `add02374`, and both matched on the board.

4 Results

4.1 Resources

nextpnr place-and-route reports for the GW1NR-9 (8640 LUT4, 6480 DFF, 26 BSRAM):

Phase	LUT4	DFF	BSRAM	$F_{\max}$ (MHz)
1 LED counter	33	–	0	239
2 UART echo	119	50	0	141
3 CPU + UART	4113 (47%)	1327	8	–
4 + accelerator	4019 (46%)	1455	8	53

Phase 4 also uses 850 ALU blocks (13%) and 40 RAM16SDP4 (14%); no hardware multipliers are used. The CPU uses more logic than the 2000–3500 LUT4 first estimated, because the multiplier, divider and compressed-instruction decoder are all enabled. $F_{\max}$ is the post-route figure from nextpnr, which targeted its default 12 MHz because no clock constraint is given; every design clears the real 27 MHz clock by a wide margin. The Phase 3 value was not recorded.

4.2 Accelerator benchmark

Measured on the board with `bench 1000`, using the CPU `rdcycle` counter:

	Cycles	Per step
Software loop	50015	50
Accelerator (including MMIO)	1066	1.07
Reduction	46×, results identical	

The accelerator itself takes one cycle per step, so about 66 of the 1066 cycles are bus overhead (three register writes, a polling loop and a read). This suggests a speedup of roughly $50N/(N + 66)$ for N steps, which would be below $1\times$ for single steps and approach $50\times$ for long runs. This is a model fitted to one measurement, not measured at other N.

5 Problems met on the way

- **Linux ModemManager corrupted the console.** After each load it probed the USB serial port and the board received its own banner and prompts as typed input, so a line such as `cycles` arrived as `> cycles` and was rejected. Fixed with a udev rule that sets `ID_MM_DEVICE_IGNORE=1` for the FTDI device.
- **Backspace bug.** A Backspace on an empty line was stored as a character. The simulation test now sends one Backspace first (two would cancel each other out and hide the bug).
- **Failed flash verification.** One `flash` reported a CRC failure and the board then misbehaved after power-up; reflashing a known-good image succeeded. Loading to SRAM with `prog` is the safe default while developing.
- **Serial port disappears on programming.** Each JTAG load re-enumerates the USB device, so a terminal opened before loading goes stale. Open the terminal after loading.
- **Simulation versus board.** The testbench initially passed with a broken design because of a race on the start signal and a string truncated by an undersized vector; both were test bugs found by checking that the test fails on bad input.

6 Limitations and next steps

The UART receive FIFO holds 64 bytes and drops the newest byte on overflow. `poke` and `peek` do no address checking, so a write into RAM can crash the program. Possible extensions: a MicroSD SPI controller, an HDMI/DVI frame buffer, or a small scheduler on the CPU. None of these is started.